

|                                                                                                                                                                                                                                                  |           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1.0 Instructions for running assignment on DE2-115 dev board.....</b>                                                                                                                                                                         | <b>2</b>  |
| 2.0 Solution Description.....                                                                                                                                                                                                                    | 3         |
| 2.1 Solution overview:.....                                                                                                                                                                                                                      | 3         |
| 2.2 Task Descriptions:.....                                                                                                                                                                                                                      | 3         |
| FrequencyAnalysisTask (Priority - 4):.....                                                                                                                                                                                                       | 3         |
| vLoadControlTask (Priority: 3):.....                                                                                                                                                                                                             | 3         |
| vSwitchPollTask(Priority: 2):.....                                                                                                                                                                                                               | 3         |
| This task periodically reads the systemState to determine user-requested load enable conditions. It polls the switch inputs at a fixed rate and updates the corresponding load enable flags while ensuring thread-safe access using mutexes..... | 3         |
| vCheckButtonTask(Priority: 2) :.....                                                                                                                                                                                                             | 4         |
| 2.3 Interrupt Service Routines:.....                                                                                                                                                                                                             | 4         |
| FrequencyRelayISR:.....                                                                                                                                                                                                                          | 4         |
| Button_interrupts_function:.....                                                                                                                                                                                                                 | 4         |
| 2.4 Helper Function Descriptions:.....                                                                                                                                                                                                           | 5         |
| isUnstable:.....                                                                                                                                                                                                                                 | 5         |
| updateLEDs:.....                                                                                                                                                                                                                                 | 5         |
| shedLowestPriorityLoad.....                                                                                                                                                                                                                      | 5         |
| reconnectHighestPriorityLoad.....                                                                                                                                                                                                                | 5         |
| allLoadsReconnected.....                                                                                                                                                                                                                         | 5         |
| SetUpMisc.....                                                                                                                                                                                                                                   | 5         |
| SetUpISRs.....                                                                                                                                                                                                                                   | 5         |
| 2.5 Design diagram.....                                                                                                                                                                                                                          | 6         |
| <b>Changes:.....</b>                                                                                                                                                                                                                             | <b>8</b>  |
| <b>3.0 Justification of design decisions.....</b>                                                                                                                                                                                                | <b>8</b>  |
| 3.1 Periodic and Non periodic task implementations.....                                                                                                                                                                                          | 8         |
| 3.2 Why did you split functionality a certain way?.....                                                                                                                                                                                          | 8         |
| 3.3 Why did you choose a specific mechanism for protecting shared variables?.....                                                                                                                                                                | 9         |
| 3.4 Why did you choose a specific communication mechanism?.....                                                                                                                                                                                  | 9         |
| Semaphores were used for event notification such as buttonSem, and loadSem. loadSem is triggered in FrequencyAnalysisTask when the state is changed to managing, and it wakes up LoadControlTask.                                                |           |
| Similarly, buttonSem is triggered in button_intercept_isr and it wakes up CheckButtonTask.....                                                                                                                                                   | 9         |
| 3.5 Why do your tasks have the priorities you've assigned?.....                                                                                                                                                                                  | 9         |
| <b>4.0 FreeRTOS usage in project.....</b>                                                                                                                                                                                                        | <b>10</b> |
| 4.1 What features of FreeRTOS did you use?.....                                                                                                                                                                                                  | 10        |
| 4.2 How do your tasks interact?.....                                                                                                                                                                                                             | 11        |
| 4.3 What are potential problems with multitasking that you've overcome or prevented?.....                                                                                                                                                        | 11        |
| <b>5.0 Work partition.....</b>                                                                                                                                                                                                                   | <b>12</b> |

# 1.0 Instructions for running assignment on DE2-115 dev board

## 1. Hardware Setup

Ensure all required peripherals are connected before powering the board:

- Connect the **USB Blaster** cable to the appropriate port on the FPGA board and host computer.
- Connect the **VGA cable** to the FPGA board.
  - If using a VGA-to-HDMI converter, ensure the converter is powered.
- Connect the **PS/2 cable** (if required for input).
- Connect the **power supply** to the FPGA board.

## 2. Software Preparation

1. Download the project **.zip** file.
2. Extract the contents to a directory **without spaces in the file path**

## 3. Programming the FPGA (Quartus)

1. Open **Quartus Prime 18.1 Programmer**.
2. Add the programming file:
  - **freq\_relay\_controller.sof**
3. Under **Hardware Setup**:
  - Select **USB Blaster**.
4. Click **Start** to program the FPGA.
5. **Keep the Programmer open** after configuration (do not close it).

## 4. Running the Nios II Software (Eclipse)

1. Open **Nios II Eclipse**.
2. Set the **workspace directory** to the project's **software** subfolder.
3. Open **Run Configurations**:
  - Select **Nios II Hardware**.
4. Verify the following:
  - The correct executable is selected:  
**project1.elf**
  - Under **Target Connection**, ensure:
    - **USB Blaster (localhost)** is detected.
5. Click **Run**.

The FPGA should now be configured and the Nios II software should be successfully running on the board.

## 2.0 Solution Description

### 2.1 Solution overview:

The frequency relay controller is an embedded system implemented on an Altera DE2-115 FPGA, targeting a Nios II soft-core processor. The system monitors an input frequency and activates relay outputs when the measured frequency crosses defined thresholds. FreeRTOS provides the real-time operating environment, with concurrent tasks managing frequency measurement, threshold comparison, and relay control. Hardware interrupts drive the frequency measurement process, with ISRs signalling tasks via binary semaphores to minimise interrupt overhead. Inter-task communication is handled through a combination of mutexes protecting shared global state and queues passing measured data between producer and consumer tasks.

### 2.2 Task Descriptions:

#### **FrequencyAnalysisTask** (Priority - 4):

This task is triggered by **FrequencyRelayISR** and uses the data received from that ISR to calculate the latest Frequency and Rate of Change(ROC). This task is also responsible for forwarding the frequency value to the **VGATask**.

If after the calculation, the Frequency or the ROC are not meeting the threshold, the **systemState** is changed from Normal to Managing, and the **LoadControlTask** is triggered using a Semaphore.

#### **vLoadControlTask** (Priority: 3):

This task manages automatic load shedding and reconnection in response to detected system instability. It remains blocked until triggered by a semaphore from the **frequencyAnalysisTask**, indicating that instability has been detected and the **systemState** has been changed to Managing. Upon activation, it immediately sheds the lowest-priority load to ensure a rapid response within the required time constraints. The time at which the first load is shed is recorded to get the time response from ISR -> First Load Shed. This time captured is also used to monitor the 500ms observation window.

It uses a timed observation window (500 ms) to determine whether the system remains stable or unstable. If instability persists over this period, additional loads are shed in order of priority. If the system remains stable, previously shed loads are gradually reconnected, starting with the highest-priority loads.

The task also accounts for system state conditions. If maintenance mode is activated, all relay-controlled load shedding is overridden, and full control is returned to the user by reconnecting all loads. Once all loads are restored and stability is maintained, the **system states** is returned to normal operation.

#### **vSwitchPollTask**(Priority: 2):

This task periodically reads the **systemState** to determine user-requested load enable conditions. It polls the switch inputs at a fixed rate and updates the corresponding load enable flags while ensuring thread-safe access using mutexes.

In normal operation, the switches have full control over enabling or disabling loads. During the managing state, the task restricts user input by allowing loads to be turned off but preventing new loads from being enabled, ensuring stability control is not overridden. In maintenance mode, switches again have full control, and any relay-based load shedding is cleared to give direct user control.

After updating the load states, the task refreshes the system LEDs to reflect the current configuration.

#### **vCheckButtonTask**(Priority: 2) :

This task manages user input from a push button to toggle the system between normal and maintenance operating modes. It blocks on a semaphore that is triggered by a button press event, ensuring efficient, event-driven operation without polling.

When activated, the task safely updates the global system state using a mutex. A button press switches the system from normal to maintenance mode, or from maintenance back to normal mode. Button presses are ignored while the system is in the managing state to prevent interference with active stability control.

Status messages are printed to indicate mode transitions for debugging or user feedback purposes.

#### **vPS2Task**: (Priority: 2):

This task gets triggered by **ps2ISR** whenever a key is pressed on the PS2 board. Depending on the button pressed, the frequency/ROC is increased or decreased by 0.5. The frequency threshold is capped between 45.0 and 52.0, and ROC between 1.0 and 100.0. This is done since setting threshold beyond these values is redundant.

#### **PRVGADraw\_Task** (priority - 1):

This task polls every 10ms, and updates all the shared values using Mutexes. It displays the Frequency and ROC with respect to time, along with a dashboard that displays all the meaningful data such as: currentFrequency, currentROC, thresholds, system State, load Status, along with all the required timing datas.

Due to the complexity of this block of code, ChatGPT was used to assist with proper integration of this task with the rest of the system. This was done since running this code would often lead to bugs that would stop the whole system from running. ChatGPT was also used to assist with creating a “Ring Buffer” data structure that could be used to store the timing related data that was stored in the task.

## 2.3 Interrupt Service Routines:

#### **FrequencyRelayISR**:

This ISR is triggered by the frequency analyser hardware when a new frequency measurement is available. It reads the latest raw frequency data from the memory-mapped register and immediately forwards this value to a queue for processing by the frequency analysis task.

The ISR uses a FreeRTOS-safe queue send function to ensure correct operation within interrupt context and supports potential context switching if a higher-priority task is unblocked. This design keeps the ISR short and efficient by offloading all computation to a dedicated task, maintaining real-time responsiveness.

A timer is started in the ISR to keep track of the tick at which the ISR was triggered.

#### **Button\_interrupts\_function**:

This ISR is triggered when a push button event occurs. It first clears the interrupt by reading and resetting the edge capture register of the button PIO, ensuring that subsequent button presses can be detected correctly.

The ISR then signals the button handling task by giving a semaphore using a FreeRTOS ISR-safe function. This allows the corresponding task to process the button event outside of the interrupt context. If the unblocked task has a higher priority, a context switch is requested before exiting the ISR.

The routine is designed to be minimal and efficient, delegating all higher-level processing to a dedicated task.

### **ps2ISR:**

This ISR is triggered when a key is pressed in the ps2Board. This ISR only sends the value of the key pressed to the **ps2Task** only when the key pressed is an ASCII key.

## **2.4 Helper Function Descriptions:**

These functions are used to keep the system modular and help with debugging. Having helper functions also allows for cleaner and shorter code blocks by offloading repetitive tasks to these functions instead of in the main tasks.

### **isUnstable:**

Checks inputted frequency and ROC values against the thresholds and returns 1 if network is currently unstable

### **updateLEDs:**

Update physical LEDs from load state (call while holding loadMutex)

### **shedLowestPriorityLoad**

Shed the lowest-priority load that is currently ON (returns 1 if one was shed)

### **reconnectHighestPriorityLoad**

Reconnect the highest-priority load that was shed (returns 1 if one was reconnected)

### **allLoadsReconnected**

Returns 1 if all shed loads have been reconnected

### **SetUpMisc**

This function initializes system resources, including load control arrays, LED outputs, and all required FreeRTOS objects. It sets default load states, clears LEDs, and creates the queue, mutexes, and semaphores used for inter-task communication and synchronization

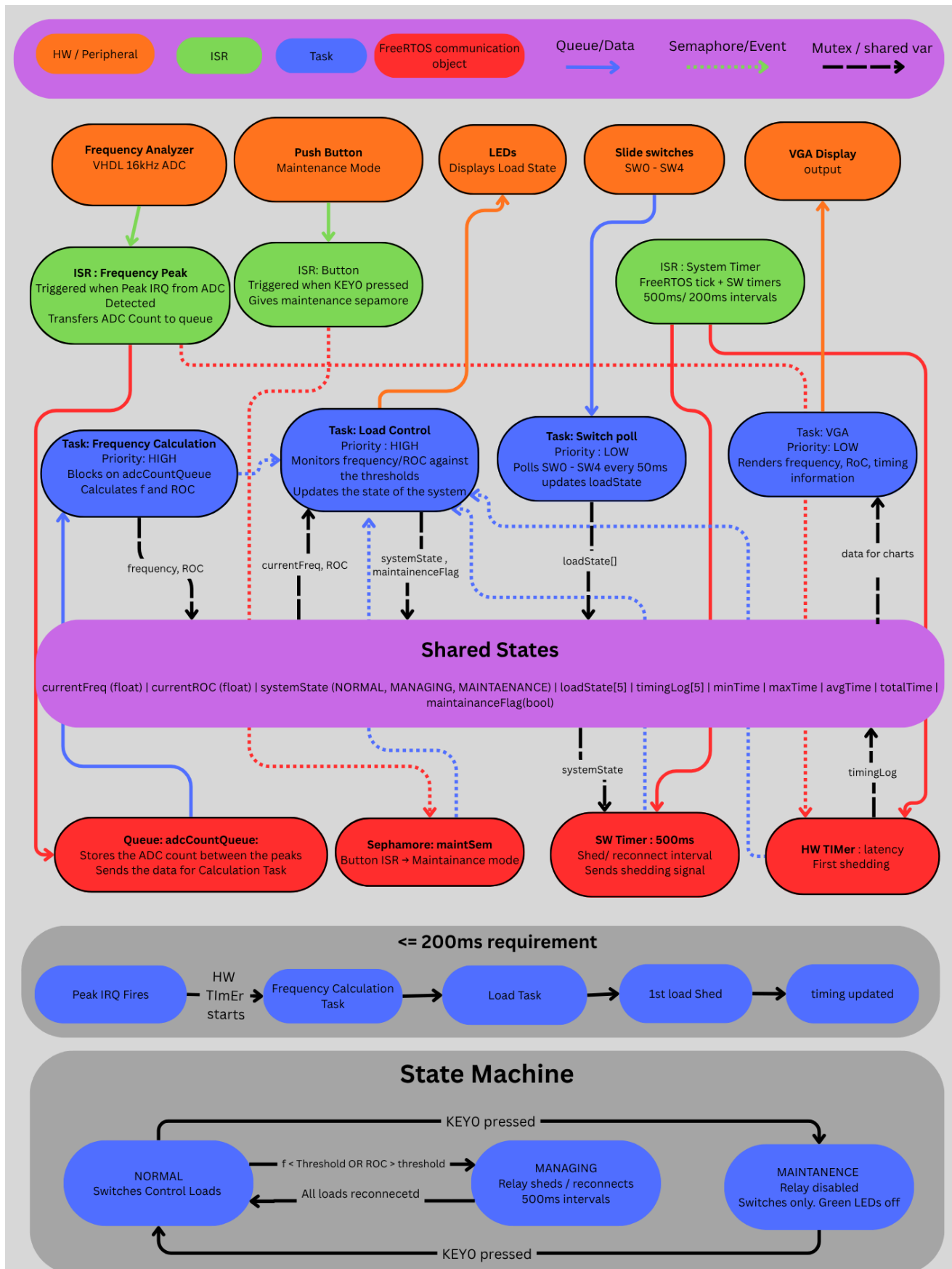
### **SetUpISRs**

This function configures and registers the system's interrupt service routines. It links the frequency analyser interrupt to its handler and enables push button interrupts by configuring the PIO interrupt mask and edge

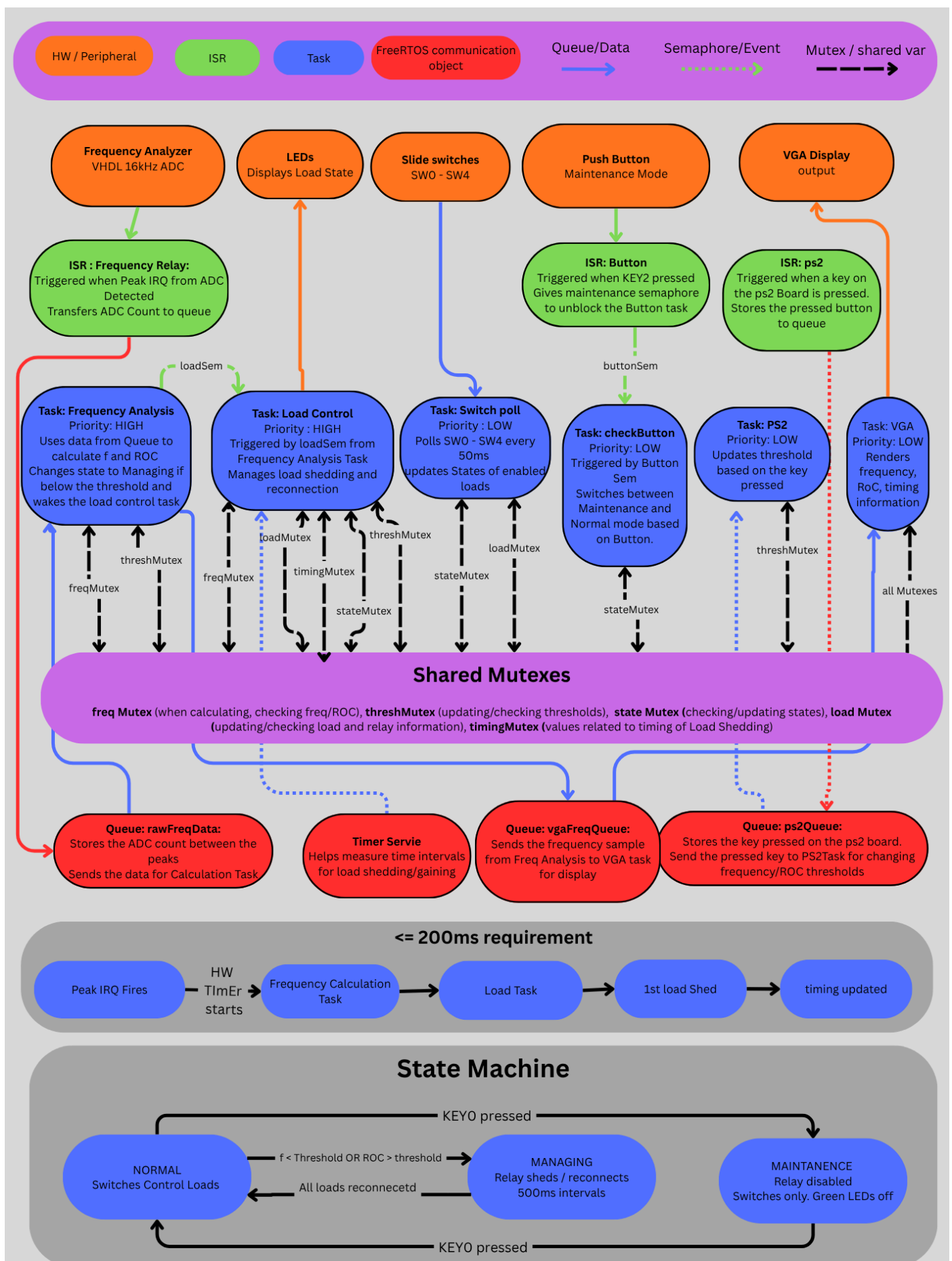
capture register. The push button ISR is then registered, allowing button events and frequency updates to be handled asynchronously.

## 2.5 Design diagram

### Initial Design diagram:



## Updated Design:



## Changes:

As a system, there has not been a massive change between how we initially thought the system would work and how it ended up being implemented. However, there are still lots of changes that were made from the initial design.

These changes from the initial design were primarily because of the novelty of FreeRTOS and a scattered understanding of how FreeRTOS works. With repeated exposure to the FreeRTOS concepts during the building of the system, the understanding of these concepts was strengthened and over time we got a clearer understanding of where we were wrong in our initial understanding.

A big difference is the number of tasks. Separate tasks were created for checking and updating the system whenever Button, or keyboard inputs were pressed. An additional queue for transferring frequency values from **FrequencyAnalysisTask** to **VGAtask** was also added. With clearer understanding of mutexes, and semaphores, we were able to add them appropriately in the final design. Although not in the design, helper functions were also added to help with modularity and a clearer code.

We initially had a very vague understanding of **Timer** in FreeRTOS, treating it like a separate ISR that would trigger at a set time which could be used by the rest of the Tasks. After a thorough look over how they work and going over the documentation, we got a better understanding and were able to integrate them properly in our project, and used the concept of ticks to keep track of time.

## 3.0 Justification of design decisions

### 3.1 Periodic and Non periodic task implementations

| Task              | Type         | Reason                   |
|-------------------|--------------|--------------------------|
| FrequencyAnalysis | Non-periodic | Driven by ISR data       |
| LoadControl       | Non-periodic | Triggered on instability |
| SwitchPoll        | Periodic     | Hardware polling         |
| ButtonTask        | Non-periodic | Interrupt-based          |
| PS2Task           | Non-periodic | Interrupt-based          |
| VGA Task          | Periodic     | Display refresh          |

### 3.2 Why did you split functionality a certain way?

Due to the complex nature of the project, and with a huge amount of things to keep track of and update as we worked on the project, we found it beneficial to decompose the task into functional units by creating separate modular blocks.

Having a separate block for Frequency Analysis, Load Control, Switch Poll, Checkin Button, PS2 INput, VGA display along with helper functions allowed us to work on smaller chunks of code which were a lot easier to individually debug as each block could be easily be tested separately for its functionality.



One downside of this was when it came to integrating separate blocks of code towards the end, some of them had to be reevaluated and rewritten to make sure that they were able to work with the other blocks of code.

### 3.3 Why did you choose a specific mechanism for protecting shared variables?

5 mutexes - **freqMutex**, **threshMutex**, **stateMutex**, **loadMutex**, **timingMutex** were used to prevent race conditions

**freqMutex** was used in FreqAnalysisTask, VGATask, and LoadControl Task whenever the value of current Frequency/ROC were to be copied or updated.

**threshMutex** was used in PS2 task, VGA task, and the helper function isUnstable() whenever the threshold values needed to be updated or read for comparison.

**stateMutex** was used in all tasks whenever the current state needed to be read or updated.

**loadMutex** was used whenever we wanted to update or read the User's intent with the loads, and Relays decision for the load such as in the helper functions ShedLowestPriority, reconnectHighestPriority, LoadControlTask, switchPollTask.

**timingMutex** was used whenever the timing information for the VGA tasks needed to be read, or updated in LoadControl task when we finished the ISR -> First shed.

### 3.4 Why did you choose a specific communication mechanism?

Semaphores were used for event notification such as buttonSem, and loadSem. loadSem is triggered in FrequencyAnalysisTask when the state is changed to managing, and it wakes up LoadControlTask.

Similarly, buttonSem is triggered in button\_intercept\_isr and it wakes up CheckButtonTask.

### 3.5 Why do your tasks have the priorities you've assigned?

The priority of the tasks was based on how important the tasks were to the system. Since the entire project relied on the frequency and ROC values, **FrequencyAnalysisTask** was given the highest priority of 4.

Since Shedding and gaining back the load required fast-reaction, **LoadControlTask** had the second highest priority of 3.

Priority of 2 was assigned to tasks that required user inputs since they are important to the project but not critical. These tasks were **ps2Task**, **switchPollTask**, **checkButtonTask**.

Finally, the **vgaTask** had the lowest priority of 1 since its only objective is to display the data and it is not critical to the functioning of the project

### 3.6 Any issues with the application design

The functional aspect which includes frequency calculation, load shedding/gaining, updates based on user input, and VGA tasks all work as required. However, the timing calculation from ISR -> First Load Shed seems to be buggy.

We seem to be getting 0ms for it, and occasionally getting 1ms which seems to be too ideal. Despite our best efforts at debugging and rearranging how we measure the time, the results remain the same. Our implementation of the ISR -> First load shed is shown below.

```
void FrequencyRelayISR(void* context, alt_u32 id)
{
    unsigned int temp = IORD(FREQUENCY_ANALYSER_BASE, 0);

    // Store the starting time when ISR occurs
    instabilityDetectedTick = xTaskGetTickCountFromISR();

    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xQueueSendToBackFromISR(rawFreqData, &temp, &xHigherPriorityTaskWoken);
    portEND_SWITCHING_ISR(xHigherPriorityTaskWoken);
}

void vLoadControlTask(void *pvParameters)
{
    while (1) {
        // Triggered when FrequencyAnalysisTask changes to managing
        xSemaphoreTake(loadSem, portMAX_DELAY);

        // shed the first load
        shedLowestPriorityLoad();

        // Time when 1st load shed
        TickType_t periodStart = xTaskGetTickCount();

        // Time between ISR -> 1st load shed
        double timeFirstShed = (double)(periodStart - instabilityDetectedTick)
* portTICK_PERIOD_MS;
.....
....
..
..
..
..
}
```

## 4.0 FreeRTOS usage in project.

### 4.1 What features of FreeRTOS did you use?

- Task Management

FreeRTOS tasks enabled the implementation of concurrent processes in the frequency relay controller.

This was accomplished using the task creation functionality of FreeRTOS, and its inbuilt scheduler and priorities systems. System functionality with hard real time requirements could be given a higher priority and therefore would execute consistently within the requirements of the system. The scheduler would then execute lower priority tasks once the core functionality had been accomplished.

- **Mutexes**  
FreeRTOS mutexes allow different tasks to effectively communicate through global variables. The global variables are critical sections of code and mutexes allow tasks exclusive access avoiding race conditions.
- **Semaphores**  
Binary semaphores were used for task synchronisation, typically for controlling non periodic tasks, certain functionality that only require execution once certain conditions are met. This prevents the overhead of running a task when it is not needed. For instance the ISR's give a semaphore that is being waited on by a task that executes the desired functionality when the interrupt occurs. This reduces the size and impact of the ISR, and runs its functionality only when needed.
- **Queues**  
Queues allow for transfer of data between Tasks and ISRs. A queue allows for reading of data in a FIFO basis in our project. This is especially useful when transferring data from FrequencyRelayISR to FrequencyAnalysisTasks for Frequency and ROC calculations, and to transfer the data of the key pressed from ps2ISR to ps2Task.
- **TickType\_t**  
TickType\_t along with the functions xgetTaskTickCount(), xgetTaskTickCountFromISR() were used along with other features for time keeping purposes and to meet the timing requirements. This was especially useful when comparing timing constraints for shedding and gaining of the loads.

## **4.2 How do your tasks interact?**

The tasks in the Frequency Relay Controller interact primarily through the FreeRTOS features detailed in 4.1.

- **Synchronisation**  
Accomplished with binary semaphores tasks can be delayed until they are needed, or the conditions for their execution are met. For instance vCheckButtonTask does not run until it receives the semaphore from the button interrupt. Until the button is pressed there is no need to run vCheckButtonTask, the conditions of execution have not changed.

A detailed description of the interaction between the Tasks, ISRs, outputs, and inputs can be seen in the Updated Diagram in 2.5.

## **4.3 What are potential problems with multitasking that you've overcome or prevented?**

To avoid race issues, mutexes were used. These ensured that 2 tasks don't access the same data at the same time. A more detailed description can be found in section 3.3. It was also ensured that when the mutexes were taken, the code blocks were small so the mutexes could be given back straight away for other tasks to utilise.

With ISRs it was ensured that they contain minimal code and low computation and were primarily used to pass on data/synchronization signals. This keeps the ISR execution fast and ensures that there are no missed interrupts.

The previous few sections go in detail how different FreeRTOS concepts were used to overcome or prevent potential problems with multitasking

## 5.0 Work partition

The work was partitioned between the 2 members - Saarthak and Jevaan.

### 1st stage:

We initially started the tasks in opposite directions intending to meet in the middle and slowly integrate our respective parts. Saarthak worked on Frequency Analysis, and Jevaan worked on handling the ps2 Inputs, and Button inputs.

This initial stage took the most time with an approximate time of **8-10 hours** on both ends since it took time to get familiar with the FreeRTOS concepts.

### 2nd stage:

After the previous stage was completed, Jevaan worked on the VGA display, along with integrating slide switches, and button pressing with a simple load shedding/gaining mechanism. Saarthak worked on building the framework of a state machine along with testing timing constraints for load shedding/gaining.

This stage was slightly faster taking approximately **6-7 hours** due to the increased familiarity with the new concepts.

### Integration:

Once a basic framework was established and independent tasks were created and tested, everything was integrated together. Saarthak worked on the integration of all tasks together while Jevaan started the documentation and report.

The integration took approximately **12-15 hours**. This was primarily because integrating independent tasks, ISRs, and other parts of the project introduced new bugs and challenges to take care of. This was not necessarily a hard part of the project, but an extremely time consuming one due to constant debugging and testing to update the code accordingly.

### VGA:

After everything was integrated, the VGA task was still slightly buggy and would often cause the entire programme to not run despite compiling. This took approximately another **3-4 hours**, after which ChatGPT was utilised heavily to ensure that it could work with the rest of the code. ChatGPT also aided in creating a ring buffer data structure that allowed for an easier storage of time related data for display.